FUNDAMENTALS IN AI & ML

PROJECT REPORT

Practical and Skills Development

CERTIFICATE

THE TASKS ENTERED IN THIS REPORT HAVE BEEN SATISFACTORILY PERFORMED BY

Registration No	: 25BAI11414
Name of Student	: ANUSHKA SHARMA
Course Name	: FUNDAMENTALS IN AI & ML
Course Code	: CSA2001
School Name	: SCAI
Slot	: C14+E11+E12
Class ID	: BL2025260500806
Semester	: WINTER 2025/26

Course Faculty Name : Dr. GARIMA JAIN

VIT BHOPAL

INDEX

S.NO	TITLE	PAGE NO.
1.	INTRODUCTION	3
2.	OBJECTIVES	4
3.	TITLE OF THE PROJECT	5
4.	METHODOLOGY	6
5.	PROJECT DESCRIPTION	8
6.	CODE	10
7.	WORKING MECHANISM	24
8.	OUTPUT	25
9.	SKILLS LEARNED	27
10.	DIFFICULTIES FACED	28
11.	CONCLUSION	29

INTRODUCTION

I wanted to pick a problem that is important and has something to do with real life as part of the Bring Your Own Project (BYOP). I didn't choose something that was purely technical; instead, I thought about problems I've seen around me. One thing that stood out to me is that a lot of people don't know about all the government welfare programs they can use. There are programs for people with disabilities, students, women, people from poor backgrounds, and people who live in rural areas. But the information is often spread out over different websites and documents, which can make it hard for a Common person to understand.

Because of this, a lot of people who deserve it either apply late or not at all. I began to consider ways to make this problem easier. That's when I realized that this situation is a lot like what we learn in **Prolog: how to make logical decisions and organize knowledge**.

We learned about Knowledge Representation in class. This is the study of how to organize information so that a computer can understand it logically. I was able to put that idea into practice with this project. I had to think about how to organize user information so that the system could reason it instead of just storing it randomly.

This Project is about using the main ideas from our Prolog syllabus to solve a real-world social problem. It helped me understand that logic programming can be used for more than just exercises in the classroom; it can also help people make decisions in real life. We learned about Proposition and First Order Logic, which is about writing logical conditions and relationships. This project is closely related to that idea because eligibility for welfare depends on certain conditions. For instance, whether someone qualifies depends on their income level, category, age, or education level. Bringing real-world situations into logical reasoning made the ideas of the syllabus much more useful. <u>So, this project is about applying the core ideas of our Prolog syllabus to solve a realistic social problem. It helped me see how logic programming can be used beyond classroom exercises and can actually support decision-making in real scenarios.</u>

OBJECTIVES

The main goal of this project was to use what we learned in Prolog to solve a real-world problem in a way that made sense and was organized.

I learned about facts, rules, operators, and backtracking while going over the syllabus. But I wanted to see how these ideas fit together when making a whole system. This project made that possible for me.

Another important goal was to be able to think logically and in a structured way. You can't just get welfare; you have to meet certain requirements. So, I had to think carefully about which factors to take into account and how they should affect the final recommendation. This required more than just writing code; it required structured thinking.

I also wanted to know how systems that reason make choices. Instead of manually checking conditions for each person, the system should automatically evaluate eligibility based on predefined logic. This directly connects to first-order logic and knowledge representation studied in the course.

I also wanted to make the system inclusive and fair. Assigning priority levels required careful thinking about which categories should receive more attention and why. This helped me understand the importance of logical decision design.

Main Objectives of the Project:

- To apply knowledge representation concepts from the syllabus.
- To use first-order logic in a real-life scenario.
- To connect Prolog theory with practical implementation.
- To develop structured and logical thinking skills.
- To design a simple rule-based decision-making system.
- To understand how reasoning systems can support social welfare decisions.

TITLE OF THE PROJECT

AI-Based Inclusive Welfare Recommendation System

The problem I am trying to solve is about making welfare schemes more accessible and easier to understand. In India, the government has initiatives to help students, people who are struggling financially, women, SC/ST communities, people living in rural areas and people with disabilities. It's not always easy to figure out which scheme you are eligible for.

Most people have to check websites to see if they qualify for a scheme. The information can be long, complicated and confusing. Sometimes people don't even know if their income level or category makes them eligible. This can lead to deserving individuals missing out on opportunities because they don't understand the eligibility criteria.

The main problem is:

How can we create a system that takes basic personal details and suggests the welfare schemes a person is eligible for? This problem is closely related to logic programming because eligibility is based on conditions. If you meet conditions then you qualify. That's how rule-based systems work.

My goal was not to replace the systems but to create a simple logical model that shows how expert systems can help with decision-making. Welfare schemes are very important. I want to make sure people can easily access them.

The welfare schemes have benefits and can really help people. I am focusing on making it easy for people to know which welfare schemes they can apply for.

This can really make a difference, in people’s lives.

METHODOLOGY / TOOLS USED

To build this project, I used Prolog as the main programming language.

Since our syllabus focuses on logic programming, Prolog was the most suitable choice. It is specifically designed for representing knowledge and writing logical rules. Instead of writing step-by-step instructions like in other programming languages, Prolog allows us to define facts and conditions, and then it automatically reasons through them.

Since our syllabus focuses on logic programming, Prolog was the most suitable choice. It is specifically designed for representing knowledge and writing logical rules. Instead of writing step-by-step instructions like in other programming languages, Prolog allows us to define facts and conditions, and then it automatically reasons through them.

The methodology I followed was:

1. First, I identified the key attributes that affect welfare eligibility, such as income, category, gender, age, education level, rural background, and disability status.
2. Then, I structured this information in a logical format so that it could be processed systematically.
3. After that, I defined different welfare schemes and their eligibility conditions based on logical reasoning.
4. I also added a priority classification system to categorize applicants based on inclusion criteria.
5. Finally, I created an interactive input system where a user can enter their details and receive recommendations.

Tools Used:

• SWI-Prolog as the programming language for implementing the logic-based system.

• Visual Studio Code (VS Code) as the development environment for writing and managing the project files.

• Integrated terminal within VS Code to execute and test the Prolog program.

• GitHub for maintaining the project repository and version control.

PROJECT DESCRIPTION

This project is a rule-based expert system developed using Prolog. The main purpose of the system is to recommend welfare schemes to users based on their personal details. The system works by collecting user information and then applying logical rules to determine eligibility.

The system considers the following attributes:

- Annual Income
- Category (SC/ST/OBC/General)
- Gender
- Education Level
- Age
- Rural Background
- Disability Status

All these details are important because most welfare schemes depend on one or more of these conditions.

The project is divided into different logical sections.

First, user data is represented in the form of structured facts. This helps in organizing the information clearly. Instead of mixing everything together, each attribute is stored separately. This reflects the concept of knowledge representation studied in the course.

Next, income is classified into categories such as low, middle, and high income. This classification helps the system apply conditions more easily. Many schemes depend on income level, so this step is important for structured reasoning.

After that, different welfare schemes are defined. Each scheme has specific eligibility conditions. For example, some schemes are meant for undergraduate students from

low-income backgrounds, while others are meant for SC/ST categories or persons with disabilities. These conditions are written in the form of logical rules.

When a user enters their details, the system checks all the defined rules. If the conditions match, the scheme is recommended. The system does not stop at just one recommendation. It continues checking and displays all schemes for which the person qualifies.

Another important feature of the project is the explanation system. Along with the scheme name, the system also displays the reason for eligibility. This makes the output more meaningful and transparent.

In addition to eligibility, I also included a priority classification system. Based on certain inclusion criteria, users are categorized into different priority levels. This reflects structured decision-making and ensures that more vulnerable categories receive higher attention.

Overall, the system works like a simple digital advisor. It takes structured input, applies logical reasoning, and provides output in an organized way. Through this project, I was able to combine knowledge representation, logical rules, and reasoning into one practical application.

CODE

```
%==========================================================
% AI-BASED INCLUSIVE WELFARE RECOMMENDATION SYSTEM
%----------------------------------------------------------
% THIS SYSTEM SUGGESTS ELIGIBLE WELFARE SCHEMES BASED ON
% USER ATTRIBUTES USING RULE-BASED REASONING
%==========================================================


%==========================================================
% USER DATA(FACTS)
%==========================================================
:- dynamic income/2.
:- dynamic category/2.
:- dynamic gender/2.
:- dynamic education/2.
:- dynamic age/2.
:- dynamic rural/2.
:- dynamic disability/2.


:- discontiguous income/2.
:- discontiguous category/2.
:- discontiguous gender/2.
:- discontiguous education/2.
:- discontiguous age/2.
```

```
:- discontiguous rural/2.
:- discontiguous disability/2.

% FORMAT:
% income(Person, level)
% category(Person, category)
% gender(Person, gender)
% education(Person, level)
% age(Person, years)
% rural(Person, yes/no)
% disability(Person, yes/no)

%==============FIRST USER====================
income(anushka, 200000).
category(anushka, obc).
gender(anushka, female).
education(anushka, undergraduate).
age(anushka, 20).
rural(anushka, yes).
disability(anushka, no).

%==========================================
%           ADDITIONAL USERS
%==========================================
```

```
income(kush, 150000).
category(kush, sc).
gender(kush, male).
education(kush, undergraduate).
age(kush, 19).
rural(kush, yes).
disability(kush, no).

income(preeti, 600000).
category(preeti, general).
gender(preeti, female).
education(preeti, postgraduate).
age(preeti, 24).
rural(preeti, no).
disability(preeti, no).

income(pranav, 120000).
category(pranav, st).
gender(pranav, male).
education(pranav, school).
age(pranav, 17).
rural(pranav, yes).
disability(pranav, yes).
```

```
%=====================================================
%            INCOME CLASSIFICATION
%=====================================================

low_income(Person) :-
    income(Person, X),
    X =< 250000.

middle_income(Person) :-
    income(Person, X),
    X > 250000,
    X =< 800000.

high_income(Person) :-
    income(Person, X),
    X > 800000.


%=====================================================
% SCHEME DEFINITIONS
%=====================================================
```

```
scheme(education_support).
scheme(women_empowerment).
scheme(sc_st_welfare).
scheme(disability_support).
scheme(rural_development).
scheme(youth_scholarship).

%-----------------------------------------------------
%_______SCHOLARSHIP ON ACCOUNT OF(REASON)______
%-----------------------------------------------------
scheme_info(education_support, 'National Education Support Scheme', 2020).
scheme_info(women_empowerment, 'Women Empowerment Initiative', 2018).
scheme_info(sc_st_welfare, 'SC/ST Welfare Development Scheme', 2016).
scheme_info(disability_support, 'Accessible India Campaign', 2015).
scheme_info(rural_development, 'Rural Development Mission', 2014).
scheme_info(youth_scholarship, 'National Youth Scholarship Program', 2019).

%============ RULE 1-EDUCATION SUPPORT ===============
eligible(Person, education_support) :-
    low_income(Person),
    education(Person, undergraduate).

%============ RULE 2-WOMEN EMPOWERMENT ===============
eligible(Person, women_empowerment) :-
```

```
    gender(Person, female),
    low_income(Person).

%============ RULE 3-SC/ST WELFARE ====================
eligible(Person, sc_st_welfare) :-
    category(Person, sc).

eligible(Person, sc_st_welfare) :-
    category(Person, st).

%============ RULE 4-DISABILITY SUPPORT ==============
eligible(Person, disability_support) :-
    disability(Person, yes).

%============ RULE 5-RURAL DEVELOPMENT ===============
eligible(Person, rural_development) :-
    rural(Person, yes),
    low_income(Person).

%============ RULE 6-YOUTH SCHOLARSHIP ================
eligible(Person, youth_scholarship) :-
    age(Person, Age),
    Age =< 21,
    education(Person, undergraduate).
```

```
%-----------------------------------------------------
% ____________SCHOLARSHIP EXPLANATIONS___________________
%-----------------------------------------------------


%-----------------------------------------------------
% EXPLAIN WHY A PERSON IS ELIGIBLE
%-----------------------------------------------------


explain(Person, Scheme) :-
    scheme_info(Scheme, Title, Year),
    write('Scheme Name: '), write(Title),
    write(' , Launched: '), write(Year), nl,
    explanation_reason(Person, Scheme), nl.


%-----------------------------------------------------
% RULE-SPECIFIC REASONS
%-----------------------------------------------------


explanation_reason(Person, education_support) :-
    low_income(Person),
    education(Person, undergraduate),
    write('Reason: Applicant belongs to economically weaker section and is pursuing
undergraduate education.').
```

```
explanation_reason(Person, women_empowerment) :-
    gender(Person, female),
    low_income(Person),
    write('Reason: Female Applicant from lower income bracket eligible under empowerment initiative. ').

explanation_reason(Person, sc_st_welfare) :-
    category(Person,sc),
    write('Reason: Applicant belongs to Scheduled Caste category as per welfare policy.').

explanation_reason(Person, sc_st_welfare) :-
    category(Person, st),
    write('Reason: Applicant belongs to Scheduled Tribe category as per welfare policy.').

explanation_reason(Person, disability_support) :-
    disability(Person, yes),
    write('Reason: Applicant qualifies under Persons with Disability(PWD) inclusion criteria.').

explanation_reason(Person, rural_development) :-
    rural(Person, yes),
    low_income(Person),
    write('Reason: Rural resident from low income background eligible under rural upliftment mission.').
```

```
explanation_reason(Person, youth_scholarship) :-
    age(Person, Age),
    Age =< 21,
    education(Person, undergraduate),
    write('Reason: Youth applicant under 21 enrolled in undergraduate program eligible for youth scholarship.').


%=====================================================
% OUTPUT SYSTEM
%=====================================================


show_eligible_yes(Person) :-
    eligible(Person, Scheme),
    write(Person), write( ' is eligible for: '),
    write(Scheme), nl,
    explain(Person, Scheme),
    write('--------------------'), nl,
    fail.


show_eligible_yes(_).


%=====================================================
```

```
% PRIORITY FOR SCHOLARSHIPS
%=====================================================

%==========HIGH================
high_priority(Person) :-
    low_income(Person),
    disability(Person, yes),
    !.

high_priority(Person) :-
    low_income(Person),
    category(Person, sc).


high_priority(Person) :-
    low_income(Person),
    category(Person, st).

%==========MEDIUM===============
medium_priority(Person) :-
    middle_income(Person),
    gender(Person, female).

%============LOW================
```

```
low_priority(Person) :-
    high_income(Person).


%-----------------------------------------------
% DISPLAY PRIORITY
%-----------------------------------------------

show_priority(Person) :-
    high_priority(Person),
    write(Person), write(' -> Priority Level: P1 (CRITICAL INCLUSION CATEGORY)'), nl,
    fail.


show_priority(Person) :-
    medium_priority(Person),
    write(Person), write( ' -> Priority Level: P2 (TARGETED WELFARE SUPPORT)'), nl,
    fail.


show_priority(Person) :-
    low_priority(Person),
    write(Person), write( ' -> Priority Level: P3 (GENERAL CATEGORY ELIGIBILITY)'),
nl,
```

```
    fail.


show_priority(Person) :-
    \+ high_priority(Person),
    \+ medium_priority(Person),
    \+ low_priority(Person),
    write(Person),
    write(' -> Priority Level: POLICY REVIEW REQUIRED'),
    nl.
show_priority(_).



%=====================================================================
====================
% RULING FUNCTION
%=====================================================================
====================


start :-
    write('======INCLUSIVE WELFARE EXPERT SYSTEM======'), nl,
    write('ENTER YOUR NAME: '), read(Name),
    ask_details(Name),
    nl, write('======ELIGIBLE SCHEMES======'), nl,
    show_eligible_yes(Name),
    nl, write('======PRIORITY STATUS======'), nl,
```

```
    show_priority(Name).



%-------------------------------------------------------
% USER CREDENTIAL SYSTEM
%-------------------------------------------------------


ask_details(Name) :-


    write('ENTER ANNUAL INCOME(numbers): '),
    read(Inc),
    retractall(income(Name, _)),
    assert(income(Name, Inc)),

    write('ENTER CATEGORY (sc/st/obc/general): '),
    read(Cat),
    retractall(category(Name, _)),
    assert(category(Name, Cat)),

    write('ENTER GENDER (male/female): '),
    read(Gen),
    retractall(gender(Name, _)),
    assert(gender(Name, Gen)),
```

```
write('ENTER EDUCATION (school/undergraduate/postgraduate): '),
read(Edu),
retractall(education(Name, _)),
assert(education(Name, Edu)),

write('ENTER AGE: '),
read(Age),
retractall(age(Name, _)),
assert(age(Name, Age)),

write('DO YOU BELONG TO A RURAL AREA? (yes/no): '),
read(Rural),
retractall(rural(Name, _)),
assert(rural(Name, Rural)),

write('UNDER PWD CRITERIA? (yes/no): '),
read(Dis),
retractall(disability(Name, _)),
assert(disability(Name, Dis)).
```

WORKING MECHANISM OF THE CODE

First, the program begins by declaring dynamic predicates. This allows the system to update user details during runtime. Since users can enter their own information, the system must be flexible enough to modify facts. This reflects how Prolog allows knowledge to be updated dynamically.

Next, user attributes such as income, category, gender, education, age, rural status, and disability status are defined. These are stored as structured facts. This step represents knowledge representation, where real-world information is converted into logical form.

After storing the user details, the program classifies income into three categories: low income, middle income, and high income. Instead of repeatedly checking numerical values in every rule, this classification simplifies decision-making. It shows how operators like greater than (>) and less than or equal to (=<) are used in Prolog.

Then, the system defines different welfare schemes. Each scheme is connected to certain eligibility conditions. These conditions are written using conjunction (AND conditions) and sometimes disjunction (OR conditions). When the program checks eligibility, it matches the user's data with these logical rules.

One important concept used here is backtracking. When checking eligibility, Prolog automatically searches through all possible rules. If one rule is satisfied, it prints the scheme and then backtracks to check if there are more matching schemes. This continues until all possibilities are explored.

The explanation mechanism works after eligibility is confirmed. It prints the name of the scheme along with a simple reason. This makes the system more transparent and understandable for the user.

The priority system works separately. Based on income level and social category, the system assigns a priority level such as P1, P2, or P3. Here, the cut operator is used to control rule execution and avoid unnecessary checking once a higher priority condition is satisfied.

Finally, the start predicate acts as the entry point of the system. When the user runs the program, they are asked to enter their details. The system then processes the input, checks eligibility, displays recommendations, and shows priority status.

Overall, the program follows a logical flow:

User Input → Knowledge Storage → Rule Matching → Backtracking → Output Generation → Priority Classification

OUTPUT SCREENSHOT

```
SWI-Prolog console
File  Settings  Tools  Debug  Help
Welcome to SWI-Prolog (threaded, 64 bits, version 10.0.1)
SWI-Prolog comes with ABSOLUTELY NO WARRANTY. This is free software.
Please run ?- license. for legal details.

For online help and background, visit https://www.swi-prolog.org
For built-in help, use ?- help(Topic). or ?- apropos(Word).

25 ?- working_directory(_, 'C:/Users/deepa/Desktop/INCLUSION EXPERT SYSTEM/').
true.

26 ?- [main].
true.

27 ?- start.
======INCLUSIVE WELFARE EXPERT SYSTEM======
ENTER YOUR NAME: anushka.
ENTER ANNUAL INCOME(numbers): |: 300000.
ENTER CATEGORY (sc/st/obc/general): |: sc.
ENTER GENDER (male/female): |: female.
ENTER EDUCATION (school/undergraduate/postgraduate): |: undergraduate.
ENTER AGE: |: 20.
DO YOU BELONG TO A RURAL AREA? (yes/no): |: no.
UNDER PWD CRITERIA? (yes/no): |: yes.

======ELIGIBLE SCHEMES======
anushka is eligible for: sc_st_welfare
Scheme Name: SC/ST Welfare Development Scheme , Launched: 2016
Reason: Applicant belongs to Scheduled Caste category as per welfare policy.
--------------------
anushka is eligible for: disability_support
Scheme Name: Accessible India Campaign , Launched: 2015
Reason: Applicant qualifies under Persons with Disability(PWD) inclusion criteria.
--------------------
anushka is eligible for: youth_scholarship
Scheme Name: National Youth Scholarship Program , Launched: 2019
Reason: Youth applicant under 21 enrolled in undergraduate program eligible for youth scholarship.
--------------------

======PRIORITY STATUS======
anushka -> Priority Level: P2 (TARGETED WELFARE SUPPORT)
true.

28 ?-
```

In this example, the user entered the following details:

- Name: anushka
- Annual Income: 300000
- Category: SC
- Gender: Female
- Education: Undergraduate
- Age: 20
- Rural Background: No
- Disability Status: Yes

After entering the details, the system processed the information and displayed all eligible welfare schemes. Based on the logical rules defined in the program, the system recommended:

- SC/ST Welfare Development Scheme
- Accessible India Campaign
- National Youth Scholarship Program

Along with each scheme, the system clearly displayed the reason for eligibility. This makes the output transparent and easy to understand.

Finally, the system assigned a priority level (P2 – Targeted Welfare Support) based on the income and category conditions.

This screenshot demonstrates that the program is working correctly. It successfully accepts user input, applies logical rules, performs backtracking to check all possible schemes, and generates structured output with explanations and priority classification.

SKILLS LEARNED

Through this project, I learned how to think more clearly and logically. While studying Prolog in class, I understood the concepts, but applying them in a full project felt different. I had to plan everything step by step.

I learned how to represent real-world information in a structured way using facts. Instead of thinking casually, I had to ask myself what information is important and how it should be stored so that the system can use it properly.

Writing eligibility rules improved my understanding of logical conditions. I became more comfortable using AND conditions, OR conditions, comparison operators, and rule checking. I also understood how Prolog searches for solutions and displays multiple outputs using backtracking.

Another important thing I learned was testing. I had to try different input combinations to check whether the system behaves correctly. This helped me become more careful while writing rules.

Overall, this project helped me improve:

- Logical and structured thinking
- Practical understanding of Prolog
- Writing condition-based rules
- Debugging small logical mistakes
- Testing different cases patiently

DIFFICULTIES FACED

While working on this project, I faced some small but important challenges. These problems were not big. They were important to solve.

At first, it was hard for me to figure out how to set up the rules in a way. I would write a condition. It would look right but the result was not what I wanted.I had to check the project to see if the problem was with the income classification or the eligibility logic in the project.

The project had another problem. I did not understand how Prolog handles what the user types in. I got errors when I tried to enter names with spaces or letters that were uppercase. But then I learned how Prolog handles atoms and variables so I understood how to enter the data for the project.

I also had some problems with setting the right directory and loading the file in SWI-Prolog for the project.

These were problems but fixing them helped me understand how the environment for the project works. They helped me learn more about the project in a practical way.

CONCLUSION

This Bring Your Own Project gave me a chance to apply what I learned in Prolog to a real-world situation. Instead of solving small classroom examples, I was able to build a complete system that takes input, applies logic, and produces meaningful output.

Through this project, I understood that knowledge representation is not just a theory. It is about organizing information in a way that a system can reason with it. I also saw how first-order logic works in practice when writing eligibility conditions.

The project made me realize that decision-making systems are built using simple logical rules. When those rules are written carefully and clearly, they can produce structured and reliable results.

I also became more confident in using Prolog. Earlier, topics like backtracking and cut felt slightly confusing. But after implementing them in this system, they became easier to understand.

Another important thing I learned is that even a simple system can solve a meaningful problem if it is designed thoughtfully. The aim was not to build something very advanced, but something logical and useful. This project helped me understand the value of clarity in design.

Overall, this experience improved my logical thinking, strengthened my understanding of the syllabus, and gave me confidence that I can apply programming concepts to practical problems.